BCS CTF

Mobile Security CTF – Walkthrough Guide

--22PC27 Ravi Varma

--22PC40 Yuthish.P

Overview

This CTF is themed around the Better Call Saul / Breaking Bad universe. Each of the 8 challenges is named after an episode and requires a different mobile security technique to extract the hidden flag. Flags follow the format flag{...} and are verified in-app. An optional Hint button is available on each card.

Tools required:

apktool, jadx-gui, adb, sqlite3, Frida, Python 3, base64

Flag verification:

SHA-256(flag + "CTF2026!")

FLAG 1: Pilot – Base64 in strings.xml

Episode: S01E01 | Category: Static Analysis | Difficulty: Easy

Prompt: Every great attorney leaves a paper trail. Jimmy McGill's first business card is hiding in plain sight inside this APK.

Location

The flag is Base64-encoded and stored in res/values/strings.xml under the key legal_disclaimer_v2.

Steps

Extract the APK:

apktool d bcs.apk -o bcs_extracted/

1. Open bcs_extracted/res/values/strings.xml and find legal_disclaimer_v2
2. The value is:
   ZmxhZ3tTQVVMX0dPT0RNQU5fQVRfTEFXfQ==

Decode:
echo "ZmxhZ3tTQVVMX0dPT0RNQU5fQVRfTEFXfQ==" | base64 -d

FLAG: flag{SAUL_GOODMAN_AT_LAW}

Screenshots

```
<string name="indeterminate">Partially checked</string>
<string name="legal_disclaimer_v2">ZmxhZ3tTQVVMX0dPT0RNQU5fQVRfTEFXfQ==
<string name="m3c_bottom_sheet_collapse_description">Collapse bottom sh
<string name="m3c_bottom_sheet_dismiss_description">Dismiss bottom shee
```

FLAG 2: Hamlin, Hamlin & McGill – Hidden Asset File

Episode: S01E02 | Category: Static Analysis | Difficulty: Easy

Prompt: Chuck always said: trust nothing you cannot verify. Somewhere inside this app, Kim hid the evidence that changes everything.

Location

The flag is stored in plaintext inside a hidden directory in the app's assets: assets/.internal_data/shadow.

Steps

1. Extract the APK:
   apktool d bcs.apk -o bcs_extracted/
2. List the assets directory (note the hidden folder .internal_data/):
   ls -la bcs_extracted/assets/
   List the hidden directory contents:
   ls bcs_extracted/assets/.internal_data/
3. Read the shadow file:
   cat bcs_extracted/assets/.internal_data/shadow
4. The flag appears on line 2 in the kim_wexler entry

```
root:*:18000:0:99999:7:::
kim_wexler:flag{KIM_IS_THE_BEST_OF_US}:19000:0:99999:7:::
chuck_mcgill:*:18500:0:99999:7:::
jimmy_mcgill:*:19200:0:99999:7:::
howard_hamlin:*:18800:0:99999:7:::
```

FLAG: flag{KIM_IS_THE_BEST_OF_US}

Screenshots

Shadow file contents showing kim_wexler entry with the flag and app submission:

```
PS D:\NEW FOLDER\apktool> .\apktool_3.0.1.jar d 'C:\Users\ASUS\Desktop\22PC40\Final APK\BCS
PS D:\NEW FOLDER\apktool>
```

FLAG 3: Fifty-One Percent – SQLite Hex Forensics

Episode: S01E03 | Category: Database Forensics | Difficulty: Medium

Prompt: Gus Fring's operation kept impeccable records. The database is corrupted — but forensics can recover the hex values hidden within.

Location

The flag is split across three hex-encoded rows in a SQLite database bundled in the app assets: assets/Laundering.db, table transactions, column hex_data.

Steps

1. Extract the APK and copy the database:
   apktool d bcs.apk -o bcs_extracted/
   cp bcs_extracted/assets/Laundering.db ./
2. Open with sqlite3:
   sqlite3 Laundering.db
3. Run the query:
   SELECT hex_data FROM transactions ORDER BY id;

4. Three rows are returned:
   666c6167
   7b39392e315f50455243454e545f
   505552455f4352595354414c7d
5. Concatenate and hex-decode using Python:
   python3 -c "print(bytes.fromhex('666c61677b39392e315f50455243454e545f505552455f4352595354414c7d').decode())"

FLAG: flag{99.1_PERCENT_PURE_CRYSTAL}

FLAG 4: Quite a Ride – Exported Activity via Intent

Episode: S01E04 | Category: Dynamic Analysis | Difficulty: Medium

Prompt: Howard Hamlin once called it 'the intent that started it all.' An exported Activity in this app is listening. Give it the right signal.

Location

ProBonoActivity is exported in the AndroidManifest. When triggered with the correct intent extra it logs a Base64-encoded flag to Logcat under the tag CTF_LEAK.

Steps

1. Open the APK in jadx-gui and confirm ProBonoActivity has android:exported="true" in the manifest
2. In jadx, open ProBonoActivity and note the required extra: hamlin_secret = "H0W4RD_S3NT_M3"
3. Trigger via adb:
   adb shell am start -n com.universe.bcs/.ProBonoActivity --es hamlin_secret H0W4RD_S3NT_M3
4. Read Logcat:
   adb logcat -s CTF_LEAK
5. The log contains:
   ZmxhZ3tIMFc0UkRfVU5MT0NLRURfVEhFX0lOVEVOVH0=
6. Decode:
   echo "ZmxhZ3tIMFc0UkRfVU5MT0NLRURfVEhFX0lOVEVOVH0=" | base64 -d

Screenshots

```
if (Intrinsics.areEqual(secretKey, "H0W4RD_S3NT_M3")) {
    Log.d("CTF_LEAK", "Hamlin Evidence Package — Exhibit A: ZmxhZ3tIMFc0UkRfVU5MT0NLRURfVEhFX0lOVEVOVH0="");
    ComponentActivityKt.setContent$default(this, null, ComposableSingletons$ProBonoActivityKt.INSTANCE.m6412ge
} else {
    ComponentActivityKt.setContent$default(this, null, ComposableSingletons$ProBonoActivityKt.INSTANCE.m6413ge
}
```

adb logcat output and app submission:

```
PS C:\Users\ASUS> adb shell am start -n com.universe.bcs/.ProBonoActivity --es hamlin_secret
Starting: Intent { cmp=com.universe.bcs/.ProBonoActivity (has extras) }
PS C:\Users\ASUS> adb logcat -s CTF_LEAK
--------- beginning of main
04-06 02:38:10.746 23187 23187 D CTF_LEAK: Hamlin Evidence Package — Exhibit A: ZmxhZ3tIMFc0U
VH0=
```

FLAG: flag{H0W4RD_UNLOCKED_THE_INTENT}

FLAG 5: Ozymandias – Frida Method Hook

Episode: S01E05 | Category: Frida Instrumentation | Difficulty: Hard

Prompt: Walter White said 99.1% is as good as perfect. Heisenberg's stash is locked behind a purity check. Hook the Vault — bypass the test.

Location

The flag is inside Vault.accessHeisenbergStash(), which is guarded by verifyMethPurity(). That method always returns false at runtime. Use Frida to hook it and force it to return true.

Steps

1. Open jadx-gui and navigate to com.universe.bcs.Vault. Confirm verifyMethPurity() returns false and accessHeisenbergStash() depends on it
2. Push and start Frida server on the device:
   adb push frida-server /data/local/tmp/

```
adb shell chmod 755 /data/local/tmp/frida-server
adb shell /data/local/tmp/frida-server &
```

3. Write the hook script (hook_vault.js):

```
Java.perform(function() {
  var Vault = Java.use('com.universe.bcs.Vault');
  Vault.verifyMethPurity.implementation = function() { return true; };
  var v = Vault.$new();
  console.log('[+] ' + v.accessHeisenbergStash());
});
```

4. Run:

```
frida -U -n com.universe.bcs -l hook_vault.js
```

5. The output contains the flag (Base64 is decoded automatically by the method)

FLAG: flag{FRIDA_HOOKS_THE_TERRITORY}

Screenshots

Frida REPL output showing hooked method and returned flag:

```
PS D:\NEW FOLDER\Frida\frida-server-17.8.2-android-x86_64> frida -U -p 231
     ____
    / _  |   Frida 17.8.2 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
   . . . .
   . . . .   Connected to Android Emulator 5554 (id=emulator-5554)
Attaching...
[+] Hook installed
[Android Emulator 5554::PID::23187 ]-> [+] verifyMethPurity hooked - retur
[+] Stash: 99.1% pure. Heisenberg approves.

Your flag: flag{FRIDA_HOOKS_THE_TERRITORY}
[+] verifyMethPurity hooked - returning true
[+] Stash: 99.1% pure. Heisenberg approves.
```

FLAG 6: Granite State – SharedPreferences XOR

Episode: S01E06 | Category: Runtime Storage | Difficulty: Medium

Prompt: Jimmy always kept a copy of everything. On first run, this app stores a legal record you can't see from the outside. Check the private storage.

Location

On first launch, SaulPrefsManager writes an XOR-encoded flag to SharedPreferences in a file called saul_legal_files. The retainer_code key holds the flag as a comma-separated list of integers, each XORed with 0x5A.

Steps

1. Install and run the app once on a rooted device or emulator
2. Pull the SharedPreferences file via adb:
   adb shell run-as com.universe.bcs \
   cat /data/data/com.universe.bcs/shared_prefs/saul_legal_files.xml
3. Locate the retainer_code string value — a comma-separated list of integers
4. XOR-decode in Python:
   vals = [60,54,59,61,33,16,19,23,23,3,5,9,19,29,20,31,30,5,14,18,31,5,8,31,14,27,19,20,31,8,39]
   print(''.join(chr(v ^ 0x5A) for v in vals))

Alternative – Frida:

Java.use('com.universe.bcs.SaulPrefsManager').revealRetainerCode()

FLAG: flag{JIMMY_SIGNED_THE_RETAINER}

Screenshots

adb shell output showing saul_legal_files.xml:

```
PS C:\Users\ASUS> adb shell run-as com.universe.bcs cat /data/data/com.universe.bcs/shared_p
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="retainer_code">60,54,59,61,33,16,19,23,23,3,5,9,19,29,20,31,30,5,14,18,31,
string>
    <boolean name="init_done" value="true" />
</map>
```

```
xor_logic.py > ...

vals= [60,54,59,61,33,16,19,23,23,3,5,9,19,29,20,31,30,5,14,18,31,5,8,31,14,2
print("".join(chr(v ^ 0x5A) for v in vals))
```

App submission and additional evidence:

```
PS D:\NEW FOLDER\Frida\frida-server-17.8.2-android-x86_64> python .\xor_logic.
flag{JIMMY SIGNED THE RETAINER}
```

```
private static final int XOR_KEY = 90;
public static final SaulPrefsManager INSTANCE = new SaulPrefsManager();
private static final int[] ENCODED = {60, 54, 59, 61, 33, 16, 19, 23, 23, 3, 5, 9, 19, 29, 20, 31, 30, 5, 14, 18, 31, 5, 8, 31, 14, 27, 19, 20, 31, 8, 39};
public static final int $stable = 8;
```

FLAG 7: Felina – BuildConfig Field

Episode: S01E07 | Category: Build Config | Difficulty: Medium

Prompt: Gale Boetticher documented everything. His lab notes were compiled into the build itself. Look in the generated config, not the source files.

Location

The flag is a buildConfigField injected into the app via build.gradle. It generates a static constant in BuildConfig.java and lives only in the compiled DEX — not in any resource file or Kotlin source.

Steps – Path A (jadx-gui)

1. Open the APK in jadx-gui
2. Navigate to com.universe.bcs.BuildConfig
3. Find the field:
   public static final String GALE_REPORT = "flag{GALE_BOETTICHER_LAB_NOTES}";

Steps – Path B (Frida)

```
Java.perform(function() {
  var BC = Java.use('com.universe.bcs.BuildConfig');
  console.log('[+] ' + BC.GALE_REPORT.value);
});
```

FLAG: flag{GALE_BOETTICHER_LAB_NOTES}

Screenshots

jadx-gui showing BuildConfig class with GALE_REPORT field:

```java
package com.universe.bcs;

/* JADX INFO: loaded from: classes3.dex */
public final class BuildConfig {
    public static final String APPLICATION_ID = "com.universe.bcs";
    public static final String BUILD_TYPE = "debug";
    public static final boolean DEBUG = Boolean.parseBoolean("true");
    public static final String GALE_REPORT = "flag{GALE_BOETTICHER_LAB_NOTES}";
    public static final int VERSION_CODE = 1;
    public static final String VERSION_NAME = "1.0";
}
```

FLAG 8: El Camino – Multi-Mechanism Static Reconstruction

Episode: S01E08 | Category: Static Analysis / Frida | Difficulty: Hard

Prompt: Gustavo Fring's distribution network was never written as a single document — it was split across couriers. The code exists in this app the same way: assembled from five separately encrypted pieces.

Location

The LosPollos class assembles the flag from five fragments stored using five different encoding mechanisms inside the companion object static initialiser. No single string literal holds the full flag.

Steps – Path A (jadx-gui)

1. Open jadx-gui and navigate to com.universe.bcs.LosPollos
2. Expand the Companion inner class and find five fields:

- PART_A: Base64.decode("ZmxhZ3s=") → "flag{"
- PART_B: intArrayOf(83,80,76,64) each XOR 0x1F → "LOS_"
- PART_C: charArray("_SOLLOP") reversed → "POLLOS_"
- PART_D: hex string "4449535452494254494f4e5f" decoded → "DISTRIBUTION_"
- PART_E: ROT13("PBQR}") → "CODE}"

3. Decode each part and concatenate in order: PART_A + PART_B + PART_C + PART_D + PART_E
4. Python decode script:

```
import base64
partA = base64.b64decode('ZmxhZ3s=').decode()
partB = "".join(chr(x ^ 0x1F) for x in [83,80,76,64])
partC = "_SOLLOP"[::-1]
partD = bytes.fromhex("4449535452494254494f4e5f").decode()
partE = "".join(chr((ord(c)-65+13)%26+65) if c.isupper() else c for c in "PBQR}")
print(partA + partB + partC + partD + partE)
```

Steps – Path B (Frida)

```
Java.perform(function() {
  var LP = Java.use('com.universe.bcs.LosPollos');
  console.log('[+] ' + LP.getDistributionCode());
});
```

FLAG: flag{LOS_POLLOS_DISTRIBUTION_CODE}